Damn Fast DFIR — Splunk SPL Cheat Sheet

by MatthySoft

Search syntax, commands, eval and stats functions, regex, time maths, DFIR hunts for Windows, Sysmon, Zeek and Linux.

OFFLINE FIELD REFERENCE
A4 print edition
20 Oct 2026
src: src/splunk_content.html

BASICS Search anatomy

```
index=zeek sourcetype=bro:conn:json id.resp_p=443   ← base search: filter here
| stats sum(orig_bytes) AS out BY id.orig_h id.resp_h  ← transform
| where out > 10000000                                ← post-filter
| sort - out | head 20                                 ← present
```

- Filter as early as possible: **index, sourcetype, time, then literal terms**, before the first pipe. Everything after a pipe runs on what the base search returned.
- Field **names** are case-sensitive. Field **values** in the base search are not (`user=ADMIN` also matches `admin`). `where` and `eval` comparisons **are** case-sensitive.
- Commands are case-insensitive. Boolean operators must be UPPERCASE: `AND OR NOT`.

BASICS Matching & boolean logic

SYNTAX	MEANING
`error fail*`	both terms (AND is implied)
`(a OR b) NOT c`	grouping; NOT applies to the next term
`"failed password"`	exact phrase
`user=adm*`	wildcard (avoid a leading `*`: slow)
`user!=admin`	field exists and is not admin
`NOT user=admin`	not admin **or field missing**
`EventCode IN (4624,4625,4648)`	any of the list (wildcards allowed)
`bytes>1000  count<=5`	numeric comparison
`TERM(10.1.1.5)`	exact indexed token, including dots (fast)
`CASE(Admin)`	case-sensitive term
`path="C:\\Windows\\*"`	backslash is escaped as `\\`
`msg="say \"hi\""`	escaped quote
`sourcetype=bro:*:json`	wildcard on metadata
`index=zeek OR index=wineventlog`	several indexes

BASICS Time modifiers

SYNTAX	MEANING
`earliest=-24h latest=now`	last 24 h (overrides the time picker)
`earliest=-7d@d`	7 days ago, snapped to midnight
`earliest=@d latest=now`	today so far
`earliest=-1d@d latest=@d`	all of yesterday
`earliest=@w1`	since Monday (`@w0` = Sunday)
`earliest=-1mon@mon latest=@mon`	last calendar month
`earliest="10/19/2026:08:00:00"`	absolute, `%m/%d/%Y:%H:%M:%S`
`earliest=0`	all time
`_index_earliest=-1h`	by **index** time (what was just ingested)

Units: s m h d w mon q y. Snap with @unit; offset after the snap: `-1d@d+8h` = 08:00 yesterday.

GOTCHAS Field names with dots & quoting

- In `eval` / `where`, **single quotes** mean a field name and **double quotes** mean a string: `| where 'id.resp_p'=443 AND proto="tcp"`.
- Unquoted dotted names work in the base search and in `stats ... BY id.orig_h`.
- Rename early to keep things simple: `| rename id.orig_h AS src, id.resp_h AS dest, id.resp_p AS dport`.
- `| rename "Account Name" AS acct`: names with spaces need double quotes in commands.
- Null maths gives null: `| fillnull value=0 orig_bytes resp_bytes` before adding.
- `sort` stops at 10,000 rows by default. Use `sort 0 field` for all rows.
- Subsearches return at most 10,000 rows and time out after 60 s by default.

EDITOR Search bar, comments, modes

KEYS / SYNTAX	DOES
`Ctrl / ⌘ + \`	auto-format the search, one command per line

EDITOR Search bar, comments, modes (cont.)

KEYS / SYNTAX	DOES
`Ctrl / ⌘ + Shift + E`	expand macros (preview the full search)
`Shift + Enter`	new line in the search bar
```` ```comment``` ````	inline comment (Splunk 8.1+)
`Fast / Smart / Verbose`	search mode. Verbose keeps events plus fields for transforming searches.
`Job › Inspect Job`	timings, scanned vs returned, subsearch output

COMMANDS Filter, shape & present

COMMAND	DOES
`| search user=admin*`	filter later in the pipeline (wildcards OK)
`| where count>10 AND like(user,"adm%")`	filter with eval expressions
`| regex CommandLine="(?i)-e(nc|ncodedcommand)\s"`	keep matching events (`!=` drops them)
`| table _time host user src`	columns, in this order
`| fields - _raw punct`	remove fields (`+` keeps only those)
`| rename src_ip AS src, "*_ip" AS *`	rename, wildcards allowed
`| dedup user host sortby -_time`	first event per combination
`| sort 0 -count +user`	sort desc / asc, all rows
`| head 20  | tail 20  | reverse`	first / last N, flip order
`| fillnull value="-" user`	replace nulls
`| eval x=..., y=...`	compute fields (see eval functions)
`| rex field=_raw "user=(?<user>\S+)"`	regex field extraction
`| spath input=json path=a.b{}.c output=c`	pull from JSON / XML
`| convert ctime(first) ctime(last)`	epoch to readable
`| fieldformat bytes=tostring(bytes,"commas")`	display format, keeps the value
`| addtotals  | addcoltotals`	row / column totals
`| transpose 0`	swap rows and columns
`| iplocation dest`	City / Country / lat / lon (offline DB)

COMMANDS Aggregate & chart

COMMAND	DOES
`| stats count dc(user) AS users values(user) BY src`	aggregate, events gone
`| eventstats avg(bytes) AS avg BY host`	add the aggregate to every event
`| streamstats count AS n current=f last(_time) AS prev BY host`	running stats in event order
`| timechart span=1h count BY host limit=10 useother=f`	time series
`| chart count OVER src BY dest_port`	2-D table
`| top limit=20 user  | rare limit=20 process`	most / least common, with %
`| bin _time span=5m`	bucket times before `stats ... BY _time`
`| xyseries row col val  | untable row col val`	long to wide / wide to long
`| tstats count WHERE index=* BY index sourcetype`	indexed-field stats (very fast)
`| transaction src maxspan=30m maxpause=5m`	group into sessions (heavy; prefer stats)
`| fieldsummary maxvals=5`	every field, its count and top values
`| anomalydetection  | cluster showcount=t`	odd events / group similar raw text

COMMANDS Combine, generate, lookup, output

COMMAND	DOES
`index=a [search index=b bad=1 | fields src]`	subsearch: its rows become `(src=x OR src=y)`

COMMANDS Combine, generate, lookup, output (cont.)

COMMAND	DOES
`[| inputlookup iocs.csv | rename ioc AS dest | fields dest]`	IOC list as a filter
`| append [search ...]`	add another result set below
`| appendpipe [stats count AS total]`	summary row from the current results
`| join type=left user [search ...]`	SQL-like join (limited; prefer stats BY)
`(index=a) OR (index=b) | stats ... BY key`	the fast "join"
`| multisearch [search ...] [search ...]`	run streaming searches together
`| map search="search index=x host=$host$" maxsearches=20`	run a search per row
`| makeresults count=3 | eval x=random()`	test data / regex playground
`| inputlookup allow.csv`	read a lookup
`| lookup allow.csv domain OUTPUT verdict`	enrich; `| where isnull(verdict)`
`| outputlookup case42_hits.csv`	save results as a lookup
`| outputcsv case42.csv`	to `$SPLUNK_HOME/var/run/splunk/csv/`
`| collect index=case42_notable`	save results into an index
`| mvexpand dest | makemv delim="," f | nomv f`	split multi-value into rows / make / flatten
`| foreach *_bytes [eval <<FIELD>>=round('<<FIELD>>'/1024)]`	same eval over many fields

FUNCTIONS eval: conditional, text, maths

FUNCTION	EXAMPLE / RESULT
`if(c, a, b)`	`if(bytes>1e6,"big","small")`
`case(c1,v1, c2,v2, true(),v)`	multi-branch; `true()` = default
`coalesce(a, b, c)`	first non-null
`isnull(x) isnotnull(x) null()`	null tests / set null
`like(x,"adm%")`	SQL-style: `%` any, `_` one char
`match(x,"(?i)^adm")`	regex test (true/false)
`cidrmatch("10.0.0.0/8", ip)`	IP in subnet
`in(x,"a","b")`	value in list
`searchmatch("error user=x")`	search syntax inside eval
`len lower upper trim ltrim rtrim`	text basics; `trim(x,"\"")`
`substr(x, start, len)`	1-based; `substr(x,-4)` = last 4
`replace(x, regex, new)`	`replace(u,"^.*\\\\","")` strips DOMAIN\
`split(x,".")  mvindex(split(x,"."),-1)`	split to multi-value; last item
`urldecode(x)  printf("%05d",n)`	decode %xx; format
`tostring(n,"commas"|"hex"|"duration")`	1,234 / 0x4D2 / 00:20:34
`tonumber(x, 16)`	parse with base (hex to dec)
`md5 sha1 sha256 sha512`	hash a string
`round(x,2) floor ceiling abs pow log ln exp sqrt`	maths
`min(a,b) max(a,b) random()`	row-level min/max
`typeof(x)`	Number / String / Boolean / Invalid
`json_extract(j,"a.b") json_object() json_keys(j)`	JSON (8.1+)
`spath(j,"a.b{}")`	JSON path inside eval
`ipmask("255.255.255.0", ip)`	mask an IP to its /24

FUNCTIONS eval: multi-value & time

FUNCTION	EXAMPLE / RESULT
`mvcount(f)`	number of values
`mvindex(f, 0)  mvindex(f,-1)`	first / last value (0-based)
`mvjoin(f, ";")`	to a single string
`mvfilter(match(f,"^10\."))`	keep matching values
`mvdedup(f)  mvsort(f)`	unique / sorted
`mvappend(a, b, "x")`	combine into multi-value
`mvfind(f, "regex")`	index of first match
`mvzip(a, b, "=")`	pair values element-wise
`mvmap(f, lower(f))`	apply to every value (8.0+)
`mvrange(0, 10, 2)`	0 2 4 6 8

FUNCTIONS eval: multi-value & time (cont.)

FUNCTION	EXAMPLE / RESULT
`now()  time()`	search start / wall clock (epoch)
`strftime(_time,"%F %T")`	epoch to text
`strptime(ts,"%Y-%m-%dT%H:%M:%S.%3N%z")`	text to epoch
`relative_time(now(),"-1d@d")`	time-modifier maths in eval
`_time - relative_time(_time,"@d")`	seconds since midnight (off-hours checks)
`strftime(_time,"%H")` `strftime(_time,"%a")`	hour / weekday

FUNCTIONS stats / chart / timechart functions

FUNCTION	RETURNS
`count  count(f)`	events / events where f exists
`count(eval(action="fail"))`	conditional count (needs AS)
`dc(f)  estdc(f)`	distinct count / estimate
`sum avg min max range`	numbers
`median mode stdev var`	distribution
`perc95(f)  p99(f)` `exactperc99(f)`	percentiles
`values(f)  list(f)`	unique sorted / all in order (list caps at 100)
`earliest(f) latest(f)`	value at the oldest / newest event
`earliest_time(f) latest_time(f)`	epoch of those events
`first(f) last(f)`	first / last as seen (newest first by default)
`min(_time) max(_time)`	first / last seen
`sparkline(count, 1h)`	mini trend per row
`per_second(f)  rate(f)`	rates (timechart / counters)

REFERENCE strftime / strptime codes

`%Y %y`	2026 / 26	`%H %I %p`	24 h / 12 h / AM-PM
`%m %b %B`	10 / Oct / October	`%M %S`	minute / second
`%d %e`	day 09 / 9	`%3N %6N`	ms / µs subseconds
`%j`	day of year 001-366	`%s`	epoch seconds
`%a %A %w`	Mon / Monday / 0-6 (Sun=0)	`%z %Z`	+0200 / CEST
`%V`	ISO week number	`%T`	%H:%M:%S
`%F`	%Y-%m-%d	`%c`	locale date + time

```
# forensic timestamp conversions to epoch
| eval t=(filetime/10000000)-11644473600     # Windows FILETIME
| eval t=(chrome/1000000)-11644473600        # Chrome / WebKit
| eval t=mac+978307200                       # Mac absolute / Cocoa
| eval t=ms/1000                             # Unix ms
| eval ts=strftime(t,"%F %T")
```

REGEX rex & regex recipes

```
| rex field=_raw "from (?<src>\d{1,3}(?:\.\d{1,3}){3}) port (?<sport>\d+)"
| rex max_match=0 "(?<ip>\b\d{1,3}(?:\.\d{1,3}){3}\b)"   # all hits as a multi-value
| rex field=url "^https?://(?<domain>[^/:]+)"
| rex field=domain "(?<sld>[^.]+\.[^.]+)$"                # registered domain (rough)
| rex field=user "^(?:[^\x5c]+\x5c)?(?<acct>.+)$"         # DOMAIN\user -> user
| rex mode=sed field=cmd "s/\s+/ /g"                      # sed-style rewrite
| regex query="^[a-z0-9]{20,}\."                          # keep long random labels
```

PATTERN	MATCHES
`\b(?:\d{1,3}\.){3}\d{1,3}\b`	IPv4
`\b[a-fA-F0-9]{32}\b  {40}  {64}`	MD5 / SHA-1 / SHA-256
`[\w.+-]+@[\w-]+\.[\w.-]+`	email address
`(?i)[a-z]:\x5c[^"\s]+\.exe`	Windows EXE path (`\x5c` = backslash)
`[A-Za-z0-9+/]{40,}={0,2}`	long base64 blob
`(?i)`	case-insensitive flag (PCRE)

Test safely: `| makeresults | eval _raw="sample text" | rex ...`. `\x5c` avoids SPL-versus-regex backslash escaping problems.

PATTERNS Analyst patterns

```
# first / last seen per thing
```

PATTERNS Analyst patterns (cont.)

```
| stats count min(_time) AS first max(_time) AS last BY user src
| convert ctime(first) ctime(last)
# new in the last 24 h vs the 30-day baseline
earliest=-30d | stats min(_time) AS first BY process
| where first >= relative_time(now(),"-24h")
# percentage of total
| stats count BY dest | eventstats sum(count) AS total
| eval pct=round(100*count/total,2) | sort - pct
# recent vs baseline side by side
| eval
    period=if(_time>=relative_time(now(),"-1d"),"recent","baseline")
| chart count OVER user BY period | where baseline=0 OR
    isnull(baseline)
# off-hours activity (before 07:00 / after 19:00 / weekend)
| eval h=tonumber(strftime(_time,"%H")), d=strftime(_time,"%a")
| where h<7 OR h>=19 OR d="Sat" OR d="Sun"
# super timeline across sources
(index=wineventlog OR index=sysmon OR index=zeek) host=WS042 OR
    src=10.1.1.42
| eval what=coalesce(CommandLine, query, uri, EventCode)
| table _time index sourcetype host user what | sort 0 _time
# outlier: more than 3 standard deviations above a host's average
| bin _time span=1h | stats count BY _time host
| eventstats avg(count) AS a stdev(count) AS s BY host
| where count > a+3*s
```

DISCOVER What data do I have?

```
| eventcount summarize=false index=* | sort - count
| tstats count WHERE index=* BY index sourcetype
| tstats min(_time) AS first max(_time) AS last WHERE index=* BY
    index sourcetype
  | convert ctime(first) ctime(last)
| metadata type=hosts index=wineventlog | convert ctime(lastTime)
| metadata type=sourcetypes index=*
index=zeek sourcetype=bro:conn:json | fieldsummary maxvals=5
| rest /services/data/indexes | table title currentDBSizeMB
    totalEventCount
| rest /services/saved/searches | table title search
index=_internal sourcetype=splunkd log_level=ERROR | stats count BY
    component
index=_audit action=search info=granted | table _time user search
```

INGEST Offline evidence import & export (CLI)

```
S=/opt/splunk/bin/splunk          # Windows: "C:\Program
    Files\Splunk\bin\splunk.exe"
$S add index case42
$S add oneshot /cases/42/zeek/conn.log -index case42 -sourcetype
    bro:conn:json
$S add oneshot /cases/42/hayabusa.jsonl -index case42 -sourcetype
    _json
$S add oneshot /cases/42/timeline.csv -index case42 -sourcetype csv
$S add oneshot /cases/42/auth.log -index case42 -sourcetype
    linux_secure -host web01
$S search 'index=case42 | stats count BY sourcetype' -output csv -
    maxout 0 > out.csv
$S list index
$S btool props list bro:conn:json --debug    # which config wins
$S stop; $S clean eventdata -index case42 -f; $S start   # empty an
    index
```

- Built-in sourcetypes for evidence: `_json` (one JSON object per line), `csv` (header row), `access_combined`, `syslog`, `linux_secure`.
- Check `_time` after import. If it shows import time, set TIMESTAMP_FIELDS / TIME_FORMAT in props.conf for that sourcetype and import again.
- The `| delete` command only hides events and needs the `can_delete` role. Use clean eventdata or a fresh index per case.

WINDOWS Field names: classic vs XML / EVTX

MEANING	XMLWINEVENTLOG / EVTX	CLASSIC WINEVENTLOG
Event ID	EventCode (EventID)	EventCode
Target user	TargetUserName	Account_Name (2nd value)
Acting user	SubjectUserName	Account_Name (1st value)
Source IP	IpAddress	Source_Network_Address
Source host	WorkstationName	Workstation_Name
Logon type	LogonType	Logon_Type
Failure code	Status / SubStatus	Status / Sub_Status
New process	NewProcessName	New_Process_Name
Command line	CommandLine	Process_Command_Line
Parent	ParentProcessName	Creator_Process_Name
Service	ServiceName / ImagePath	Service_Name / Service_File_Name

WINDOWS Field names: classic vs XML / EVTX (cont.)

The hunts below use XML/EVTX names. Sysmon names (Image, ParentImage, CommandLine, TargetFilename, DestinationIp, QueryName) are the same in both formats. Indexes and sourcetypes vary per site: run `| tstats count WHERE index=* BY index sourcetype` first.

HUNT Windows: logons & accounts

```
# brute force / spray: many failures per source
index=wineventlog EventCode=4625
| stats count dc(TargetUserName) AS users values(TargetUserName) AS
    names BY IpAddress
| where count>20 OR users>10 | sort - count
# failures followed by a success (same source + user)
index=wineventlog EventCode IN (4624,4625)
| stats count(eval(EventCode=4625)) AS fail
    count(eval(EventCode=4624)) AS ok
  min(_time) AS first max(_time) AS last BY IpAddress TargetUserName
| where fail>5 AND ok>0 | convert ctime(first) ctime(last)
# RDP (10) and network (3) logons, with the source
index=wineventlog EventCode=4624 LogonType IN (3,10) NOT
    TargetUserName="*$"
| stats count min(_time) AS first BY TargetUserName IpAddress
    WorkstationName LogonType
# explicit credentials (runas, lateral movement)
index=wineventlog EventCode=4648
| table _time host SubjectUserName TargetUserName TargetServerName
    ProcessName
# account and group changes
index=wineventlog EventCode IN
    (4720,4722,4724,4728,4732,4738,4756,4726)
| table _time host EventCode SubjectUserName TargetUserName
    MemberSid
# audit / log cleared
index=wineventlog (EventCode=1102 OR EventCode=104) | table _time
    host SubjectUserName
# Kerberoasting: RC4 service tickets, many SPNs per user
index=wineventlog EventCode=4769 TicketEncryptionType=0x17 NOT
    ServiceName="*$"
| stats dc(ServiceName) AS spns values(ServiceName) BY
    TargetUserName IpAddress
| where spns>3
```

4625 SUBSTATUS	MEANING
0xC0000064	user does not exist
0xC000006A	wrong password
0xC0000234	account locked out
0xC0000072	account disabled
0xC000006F / 0xC0000070	outside allowed hours / workstation
0xC0000071 / 0xC0000193	password expired / account expired

HUNT Windows: execution & persistence

```
# suspicious command lines (4688 or Sysmon 1); search terms are
    case-insensitive
(index=wineventlog EventCode=4688) OR (index=sysmon EventCode=1)
  (CommandLine="* -enc*" OR CommandLine="*FromBase64String*" OR
    CommandLine="*DownloadString*"
    OR CommandLine="*IEX*" OR CommandLine="*-w hidden*" OR
    CommandLine="*bypass*")
| eval proc=coalesce(Image, NewProcessName),
    parent=coalesce(ParentImage, ParentProcessName)
| table _time host User proc parent CommandLine
# LOLBins
index=sysmon EventCode=1 Image IN ("*\\certutil.exe",
    "*\\mshta.exe", "*\\regsvr32.exe",
  "*\\rundll32.exe", "*\\bitsadmin.exe", "*\\wmic.exe",
    "*\\msbuild.exe", "*\\installutil.exe")
| stats count values(CommandLine) AS cmds BY host Image
# Office / browser spawning shells
index=sysmon EventCode=1 ParentImage IN ("*\\winword.exe",
    "*\\excel.exe", "*\\powerpnt.exe",
  "*\\outlook.exe", "*\\chrome.exe", "*\\msedge.exe")
  Image IN ("*\\cmd.exe", "*\\powershell.exe", "*\\wscript.exe",
    "*\\cscript.exe", "*\\mshta.exe")
# rare processes across the estate
index=sysmon EventCode=1 | stats dc(host) AS hosts count BY Image |
    where hosts<=2 | sort count
# new services (7045 System / 4697 Security)
index=wineventlog EventCode IN (7045,4697)
| eval path=coalesce(ImagePath, ServiceFileName) | table _time host
    ServiceName path
# scheduled tasks created
index=wineventlog EventCode=4698 | table _time host SubjectUserName
    TaskName TaskContent
# Run keys and startup folder (Sysmon 13 / 11)
index=sysmon (EventCode=13 TargetObject="*\\CurrentVersion\\Run*")
  OR (EventCode=11 TargetFilename="*\\Start
    Menu\\Programs\\Startup\\*")
```

HUNT Windows: execution & persistence (cont.)

```
| table _time host Image TargetObject Details TargetFilename
# PowerShell script blocks (4104)
index=powershell EventCode=4104 (ScriptBlockText="*Invoke-Mimikatz*"
   OR ScriptBlockText="*Net.WebClient*"
  OR ScriptBlockText="*AmsiUtils*" OR
   ScriptBlockText="*VirtualAlloc*")
| table _time host ScriptBlockText
# LSASS access (Sysmon 10)
index=sysmon EventCode=10 TargetImage="*\\lsass.exe" NOT SourceImage
   IN ("*\\MsMpEng.exe", "*\\csrss.exe")
| stats count values(GrantedAccess) BY host SourceImage
```

HUNT Windows: Sysmon network & DNS

```
# processes making outbound connections (Sysmon 3)
index=sysmon EventCode=3
| where NOT (cidrmatch("10.0.0.0/8",DestinationIp) OR
   cidrmatch("172.16.0.0/12",DestinationIp)
  OR cidrmatch("192.168.0.0/16",DestinationIp) OR
   cidrmatch("127.0.0.0/8",DestinationIp))
| stats count dc(DestinationIp) AS ips values(DestinationPort) AS
   ports BY host Image
| sort - ips
# DNS by process (Sysmon 22)
index=sysmon EventCode=22 | stats count dc(host) AS hosts BY
   QueryName Image | sort count
# script hosts talking to the internet
index=sysmon EventCode=3 Image IN ("*\\powershell.exe",
   "*\\wscript.exe", "*\\mshta.exe",
  "*\\rundll32.exe", "*\\regsvr32.exe") | table _time host Image
   DestinationIp DestinationPort
```

HUNT Network: Zeek (dfd: index=zeek, sourcetype=bro:<log>:json)

```
# top outbound talkers by bytes
index=zeek sourcetype=bro:conn:json
| where NOT cidrmatch("10.0.0.0/8",'id.resp_h') AND NOT
   cidrmatch("192.168.0.0/16",'id.resp_h')
  AND NOT cidrmatch("172.16.0.0/12",'id.resp_h')
| stats sum(orig_bytes) AS out sum(resp_bytes) AS in count BY
   id.orig_h id.resp_h
| eval MB_out=round(out/1048576,1) | sort - out | head 25
# beaconing: regular gaps between connections (low jitter)
index=zeek sourcetype=bro:conn:json | sort 0 _time
| streamstats current=f last(_time) AS prev BY id.orig_h id.resp_h
   id.resp_p
| eval gap=_time-prev | where isnotnull(gap)
| stats count avg(gap) AS avg stdev(gap) AS sd BY id.orig_h
   id.resp_h id.resp_p
| where count>20 | eval jitter=round(sd/avg,3) | where jitter<0.15 |
   sort jitter
# long connections
index=zeek sourcetype=bro:conn:json duration>3600
| table _time uid id.orig_h id.resp_h id.resp_p proto service
   duration orig_bytes resp_bytes
# scanners: many ports or hosts, mostly unanswered
index=zeek sourcetype=bro:conn:json
| stats dc(id.resp_p) AS ports dc(id.resp_h) AS hosts
   count(eval(conn_state="S0" OR conn_state="REJ")) AS failed BY
   id.orig_h
| where ports>100 OR hosts>50
# protocol on an unexpected port
index=zeek sourcetype=bro:conn:json service=* NOT (service=http
   id.resp_p IN (80,8080))
  NOT (service=ssl id.resp_p=443) NOT (service=dns id.resp_p=53) NOT
   (service=ssh id.resp_p=22)
| stats count BY service id.resp_p | sort - count
```

HUNT Network: Zeek DNS, HTTP, TLS, files

```
# long / random DNS names (tunnelling, DGA)
index=zeek sourcetype=bro:dns:json | eval len=len(query)
| where len>50 | stats count dc(query) AS uniq BY id.orig_h | sort -
   uniq
# NXDOMAIN storms
index=zeek sourcetype=bro:dns:json rcode_name=NXDOMAIN | stats count
   BY id.orig_h | sort - count
# TXT / NULL queries, and rare registered domains
index=zeek sourcetype=bro:dns:json qtype_name IN (TXT,NULL) | stats
   count BY id.orig_h query
index=zeek sourcetype=bro:dns:json | rex field=query "(?<sld>[^.]+\.
   [^.]+)$" | rare limit=50 sld
# rare user agents (few hosts, few hits)
index=zeek sourcetype=bro:http:json
| stats count dc(id.orig_h) AS hosts values(host) AS sites BY
   user_agent | sort count
# executables over HTTP, and HTTP to bare IPs
index=zeek sourcetype=bro:http:json "application/x-dosexec"
| eval mime=mvjoin(coalesce('resp_mime_types{}', resp_mime_types),
   ", ")
| table _time id.orig_h host uri user_agent mime
```

HUNT Network: Zeek DNS, HTTP, TLS, files (cont.)

```
index=zeek sourcetype=bro:http:json | regex host="^\d{1,3}
   (\.\d{1,3}){3}(:\d+)?$"
# TLS: no SNI, old versions, failed validation
index=zeek sourcetype=bro:ssl:json (NOT server_name=* OR version IN
   (SSLv3,TLSv10)
  OR (validation_status=* NOT validation_status=ok))
| stats count BY id.orig_h id.resp_h server_name version
   validation_status
# files seen on the wire (hashes for IOC lookups)
index=zeek sourcetype=bro:files:json mime_type IN ("application/x-
   dosexec","application/x-msdownload")
| eval tx=coalesce(mvjoin('tx_hosts{}',","), 'id.resp_h'),
   rx=coalesce(mvjoin('rx_hosts{}',","), 'id.orig_h')
| table _time tx rx filename mime_type md5 sha1 sha256
```

CONN_STATE	MEANING
S0	SYN sent, no reply (scan, dead host)
REJ	SYN answered with RST (port closed)
SF	normal establish + close
S1 / S2 / S3	established, not closed / only orig closed / only resp closed
RSTO / RSTR	established, then reset by orig / resp
RSTOS0 / RSTRH	orig SYN then RST / resp SYN-ACK then RST
SH / SHR	orig / resp sent SYN+FIN only (half-open)
OTH	no SYN seen (mid-stream)

JSON arrays arrive as `field{}` (e.g. `'resp_mime_types{}'`, `'tx_hosts{}'`; Zeek 6+ files.log uses `id.*` instead of tx/rx_hosts). Zeek `history`: S syn, h syn-ack, A ack, D data, F fin, R rst, T retransmit, C bad checksum, ^ direction flipped. UPPER = originator, lower = responder.

HUNT Linux & web servers

```
# SSH brute force (sourcetype linux_secure)
index=linux sourcetype=linux_secure "Failed password"
| rex "for (?:invalid user )?(?<user>\S+) from (?<src>\S+) port"
| stats count dc(user) AS users BY src | where count>20 | sort -
   count
# successful SSH logins (password vs key)
index=linux sourcetype=linux_secure "Accepted"
| rex "Accepted (?<method>\S+) for (?<user>\S+) from (?<src>\S+)"
| stats count min(_time) AS first BY user src method | convert
   ctime(first)
# sudo commands
index=linux sourcetype=linux_secure "COMMAND="
| rex "sudo:\s+(?<user>\S+) : .*?USER=(?<as>\S+) ; COMMAND=(?
   <cmd>.*)$"
| table _time host user as cmd
# new users, groups, password changes
index=linux ("new user" OR "new group" OR "useradd" OR "usermod" OR
   "password changed")
# web: scanners and attack strings (access_combined)
index=web sourcetype=access_combined (useragent="*sqlmap*" OR
   useragent="*nikto*"
  OR useragent="*nmap*" OR uri="*../*" OR uri="*union*select*" OR
   uri="*/etc/passwd*")
| stats count values(uri) AS uris BY clientip useragent
# web shells: rare script pages hit by only a few clients
index=web sourcetype=access_combined uri_path IN ("*.php", "*.jsp",
   "*.aspx", "*.asp")
| stats count dc(clientip) AS clients values(method) AS methods BY
   uri_path
| where clients<=2 | sort count
# error spikes per client
index=web status>=400 | stats count BY clientip status | sort -
   count
```